

DAN1

FYT Final Report

Multi-modal Retrieval-Augmented Generation (RAG): Tackling Hallucination in Multimodal Question Answering

by

LAM, Ching Yin

DAN1

Advised by

Prof. Dan XU

Submitted in partial fulfillment of the requirements for CPEG 4902

in the

Department of Computer Science & Engineering

The Hong Kong University of Science and Technology

2025-2026

Date of submission: April 17, 2026

Acknowledgements

First and foremost, I am deeply grateful to my supervisor, Dr. Dan XU, for his consistent support and guidance throughout the project. His expertise in computer vision and multimodality shaped the direction of this work. I also thank the open-source community behind DINOv2 and Talk2DINO, whose publicly released models made this implementation possible. Finally, my deepest thanks go to my family and friends, for being my constant source of strength and encouragement throughout this journey.

Abstract

Retrieval-Augmented Generation (RAG) has become the major method for grounding Large Language Model (LLM) outputs with external evidence, but existing text-only pipelines cannot accommodate multimodal information. Multimodal RAG systems are being developed for use with Multimodal LLMs (MLLMs), and can process texts and images. Current systems flatten visual features into text captions, or rely on CLIP-based embeddings that prioritize semantic alignment over fine-grained visual details, limiting RAG in applications that require visual precision.

This project presents a multimodal RAG pipeline that preserves fine-grained visual features while enabling cross-modal alignment. We propose a dual-representation framework maintaining separate representation spaces, using state-of-the-art pre-trained foundational models for feature extraction and cross-modal alignment. Our contributions include: (i) a complete multimodal RAG pipeline that allows for text-only and text-image queries, with retrieved evidence covering both modalities; (ii) a method without fine-tuning requirements that leverages state-of-the-art foundational models to use in evidence retrieval; and (iii) empirical evaluation on the effect of different retrieval modes in grounding generation to retrieved evidence. We compare dense retrieval against graph-based retrieval and analyze how different retrieval modes affect grounding in MLLM generation.

Table of Contents

1	Introduction	6
1.1	Overview	6
1.2	Objectives	7
1.3	Literature Review	8
2	Methodology	13
2.1	Design	13
2.2	Implementation	15
2.3	Testing	24
2.4	Evaluation	37
3	Discussion	44
3.1	Analysis of Findings	44
3.2	Limitations	45
3.3	Future Work	46
4	Conclusion	48
A	Prompt Templates	52
B	Summary of Mathematical Notations	59
C	Retrieval and Generation Integration Tests	62
D	Project Planning	64
D.1	GANTT Chart	64
E	Required Hardware and Software	65
E.1	Hardware	65
E.2	Software	65

1 Introduction

1.1 Overview

Retrieval-Augmented Generation (RAG) has emerged as the dominant method to ground Large Language Model (LLM) generation outputs with retrieved external evidence in text-only settings [1]. Multimodal LLMs (MLLMs) capable of processing both text and visual content are becoming widely used, yet existing and widely used RAG pipelines designed for text-only models cannot directly accommodate multimodal information as retrieved evidence for grounding generation.

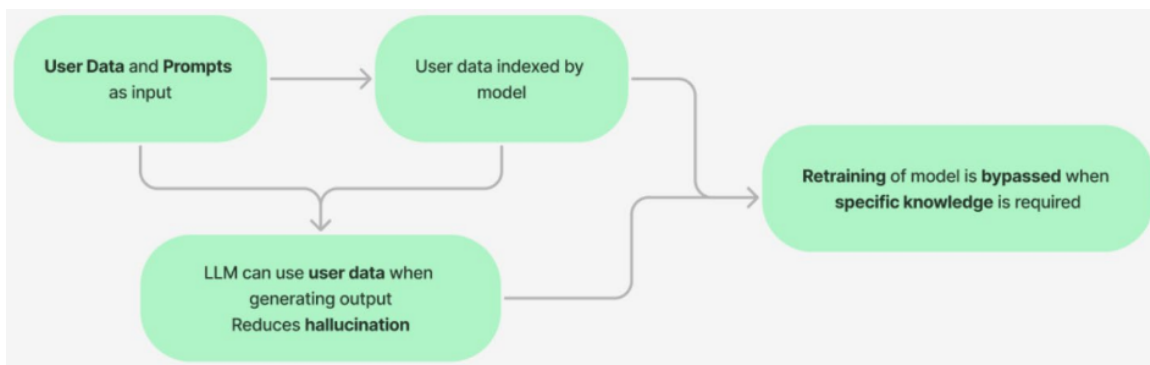


Figure 1: Rationale behind RAG

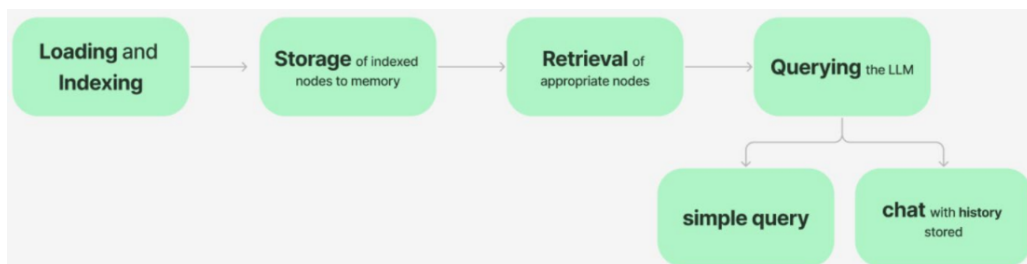


Figure 2: A typical text-based RAG pipeline

Consider a domain expert querying a MLLM on the fine-grained difference between two rare bird species, an engineer asking a MLLM about specific designs on design plans in scanned blueprint documents, or a doctor inspecting a patient’s X-ray scans for structures that may appear in certain diseases with the help of a MLLM. In these cases, relevant evidence in visual form cannot be retrieved for use with a MLLM when widely adopted text-only RAG pipelines are used. Existing multimodal RAG pipelines demonstrate bias towards text instead of visual

information and rely on captions that do not contain visual features, or may not be able to capture fine-grained visual features precisely [2, 3]. This modality gap fundamentally limits the capabilities of RAG systems, and the application of RAG in domains where visual elements dominate, as well as high-stakes domains where precision and accuracy are crucial.

Recent surveys have reviewed and documented the development of multimodal RAG and have noted advancements in graph storage structures and agentic frameworks [4]. However, limitations in existing approaches may bring limited accuracies and hinder deployment. For instance, visual-based layout cues or fine-grained details are often overlooked by the visual encoding models used. Generic cross-modal encoding models such as CLIP-based models [5] and their variants also fail to capture fine-grained visual detail; and specialized powerful vision models such as DINO [6] inherently lack a natural language interface.

Works investigating hallucinations in MLLMs [7, 8, 9] analyze underlying causes of hallucination, benchmarks and metrics for evaluation, and mitigation strategies. These surveys find that reliable grounding with retrieved visual inputs in MLLMs is still a challenging task.

1.2 Objectives

This project aims to design a RAG pipeline that effectively retrieves both textual and visual evidence to ground generation in MLLMs, while preserving necessary fine-grained visual features and elements in visual evidence. We hypothesize that visual features and semantic text features require separate representation spaces for better performance, instead of flattening both visual and textual information into one single representation space for retrieval. We propose a dual-representation framework for processing and storing external documents containing evidence for question answering as follows:

- **Textual Representations:** Original document text fused with comprehensive captions of inline images, either embedded with a neural text embedding model, or processed to form a knowledge graph of entities and relationships.
- **Visual Representations:** Obtain raw image features using DINOv2 [6], a powerful vision-only foundational model to preserve fine-grained visual features.

- **Cross-modal Connection:** Align CLIP text embeddings to the DINOv2 visual representation space using the projection layer in Talk2DINO [10], to enable text-to-image retrieval without substantial loss of visual elements due to flattening into another representation.

Our proposed pipeline supports various retrieval modes, including text-to-text, text-to-image, and image-to-image; as well as two storage and retrieval methodologies, including dense retrieval from a vector database, and knowledge graph representation of the corpus to process.

Our contributions include: (i) a complete multimodal RAG pipeline that allows for text-only and text-image queries, with retrieved evidence covering both modalities; (ii) a method without fine-tuning requirements that leverages state-of-the-art foundational models to use in evidence retrieval; and (iii) empirical evaluation on the effect of different retrieval modes in grounding generation to retrieved evidence.

1.3 Literature Review

1.3.1 Bridging Different Modalities

CLIP CLIP (Contrastive Language-Image Pre-training) [5] provides a method to represent images with natural language. Prior to CLIP, multimodal alignment largely relied on fixed-label classification, such as in ImageNet [11] and AlexNet [12], and representing images using long-form natural language was not possible.

CLIP introduces a method to bridge general natural language with images. A dual-encoder setup is trained via a contrastive loss objective to map natural language representations and image features into a shared representation space, using massive web-scale datasets of up to 400M image-text pairs as training data. This pretraining step on large-scale data allows for robust zero-shot generalization by bridging the open-vocabulary problem.

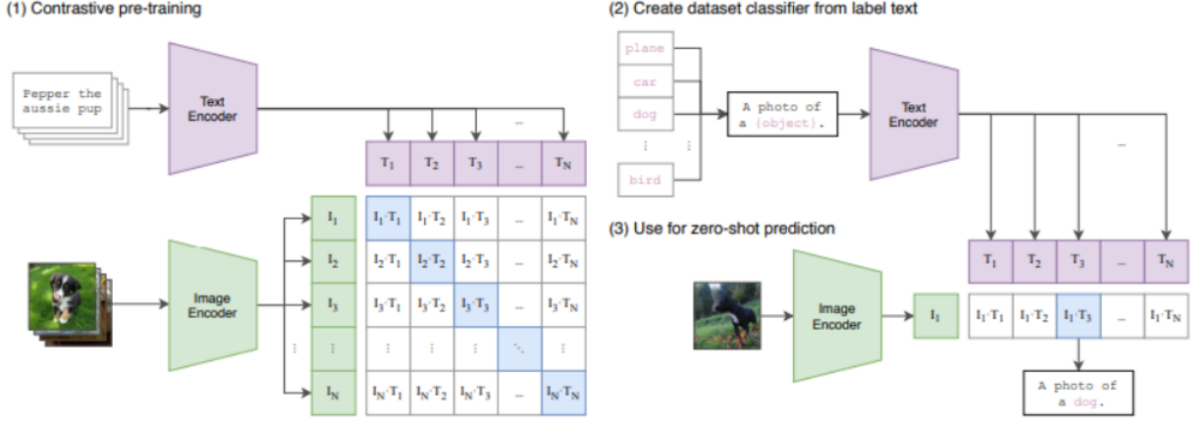


Figure 3: Embedding method proposed under the CLIP model [5]

However, the original CLIP model suffers from a limited context window size, which only allows for alignment with brief and concise captions. This lacks the capability to align representations of images and their detailed long-form captions. A later variant, Long-CLIP [13], resolves this issue by allowing the CLIP model to align to longer and more detailed natural language descriptions of images, providing more robust alignment between text and images.

DINOv2 and Talk2DINO DINOv2 [6] is a high-performing vision foundational model that can produce visual features from images. The model is a Vision Transformer (ViT) model trained on a dataset of 142M images (LVD-142M), employing a self-supervised learning method. Features generated from DINOv2 require no task-specific tuning to achieve state-of-the-art performance across various vision tasks, such as segmentation, depth estimation, and matching across images.

While DINOv2 achieves high performance in fine-grained visual tasks, DINOv2 inherently lacks a connection to natural language. Talk2DINO [10] leverages both the exceptional visual accuracy of DINOv2 and text embeddings of CLIP to fill this gap, using a non-linear mapping layer to map CLIP embeddings to the DINOv2 embedding space. This model enables state-of-the-art unsupervised open-vocabulary segmentation and produces superior segmentation masks. The vision-language alignment performance of this model also makes it possible for text-to-image retrieval, using detailed prompts and captions.

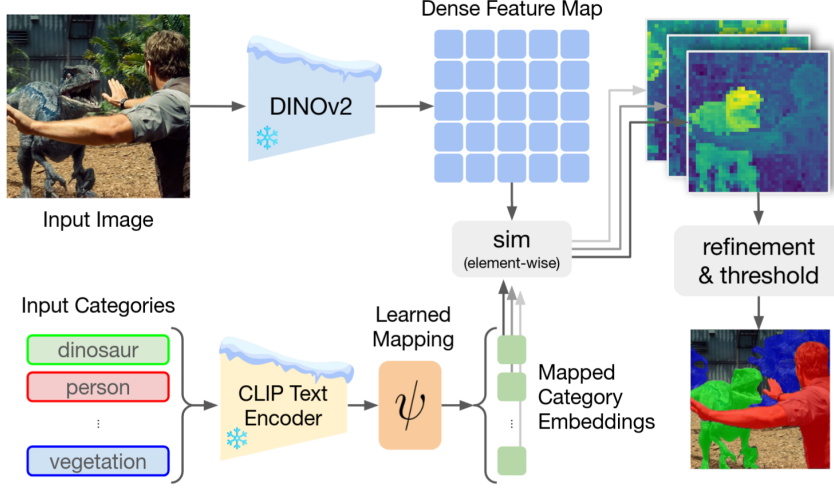


Figure 4: Overview of Talk2DINO [10]

Positioning Unlike previous works that use CLIP to embed visual elements and lose fine-grained visual elements, or rely on text-based captions that discard visual information, our pipeline uses Talk2DINO to query items in the DINOv2 embedding space using natural language, preserving fine-grained visual details while enabling text-based queries.

1.3.2 Visual-RAG Benchmarks

Visual-RAG Visual-RAG [14] focuses on text-to-image retrieval for knowledge-intensive queries, using the iNaturalist 2021 [15] dataset (with 2.7M images of 10,000 different species). Their findings revealed that noise in retrieved image batches may confuse MLLMs, especially when the query targets visual evidence only present in a small region. This motivates our preservation of raw visual features instead of flattening to image captions.

MMDocRAG MMDocRAG [16] provides expert-annotated QA pairs requiring retrieval of multi-page or multi-document cross-modal evidence. This benchmark tests end-to-end generation instead of retrieval-only evaluation.

Positioning Our work provides a complete end-to-end multimodal RAG pipeline that can be evaluated on retrieval and generation. We use Visual-RAG and MMDocRAG to evaluate retrieval accuracy and end-to-end generation for our pipeline.

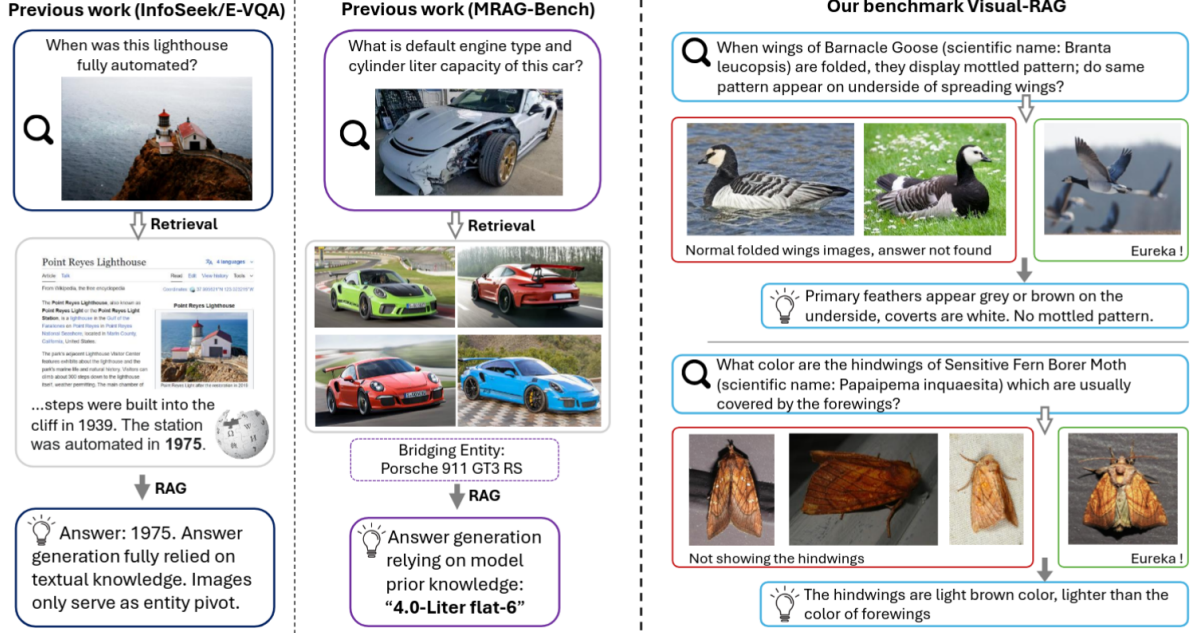


Figure 5: Comparison between previous benchmarks and Visual-RAG [14]

1.3.3 Graph-based RAG

GraphRAG GraphRAG [17] proposed using knowledge graphs for text-only RAG, where entities form nodes, relationships between entities form edges, and community summaries generated by an LLM enable global sensemaking over document collections.

VideoRAG VideoRAG [18] extended the graph-based storage method to video content. Captions of frames and video transcript are combined and transformed into a graph structure. It was also shown that their methods in handling long-context content with graph representations demonstrate superior performance compared to existing methods.

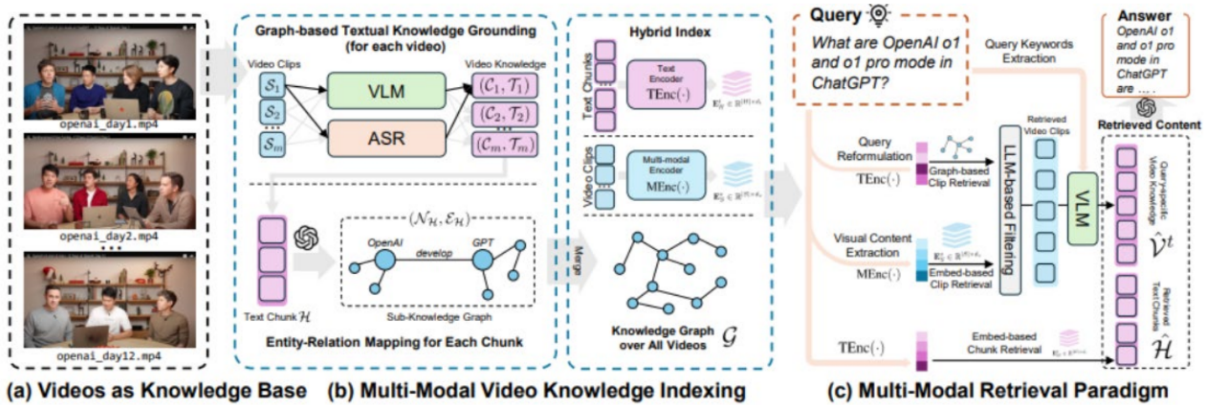


Figure 6: VideoRAG Framework for processing and retrieving video information [18]

Positioning We adapt GraphRAG’s community detection and summarization approach to our multimodal setting as an alternate configurable storage option. Image entities are integrated as nodes linked via metadata to the visual vector database, and retrieval can be performed either on dense vectors or graph communities, with configurable selection of storage methods. This allows comparison between dense retrieval and graph-based retrieval on the same pipeline.

1.3.4 Comparison with Prior Work

Feature	Text-only RAG	CLIP-based	VideoRAG [18]	Ours
Text retrieval	✓	✓	✓	✓
Image retrieval	×	✓	✓	✓
Cross-modal	×	✓	×	✓
Fine-grained visual features	—	×	×	✓
Graph-based alternative	×	×	✓	✓

2 Methodology

2.1 Design

The design of the project involves the following aspects.

2.1.1 Architecture Overview

Our proposed pipeline consists of four major modules, including the following:

Document Preprocessing We preprocess documents into the expected format before embedding and storage into the knowledge base. This step performs text splitting into overlapping chunks, and inline image extraction and context-aware captioning.

Dual-Representation Database Storage We maintain two separate indexes, with one for text chunks, and the other as a multi-vector database for image features and captions. The text index may be a vector database if dense retrieval is selected in the configurations, or a knowledge graph of entities and relationships when graph retrieval mode is chosen.

Multimodal Retrieval Retrieval takes on three modes (text-to-text, text-to-image, and image-to-image), depending on the modalities of the input query. Dense retrieval uses cosine similarity between embeddings, while graph retrieval extracts the most relevant communities in the knowledge graph.

Grounding Generation Retrieved texts and images form a final prompt with the raw query. The final prompt is then passed to a MLLM to produce a final answer conditioned on retrieved evidence, ensuring accuracy by grounding on external knowledge as context.

2.1.2 Design Principles

Our project objectives guide all pipeline decisions.

We preserve fine-grained details in each modality (text and visual), and choose methods that keep each modality processed in their respective representation spaces. By using dual

representations instead of flattening representations into a single embedding space, we allow for robust text features and fine-grained visual details to both be leveraged in retrieval and grounding generation.

Our pipeline provides a training-free multimodal RAG solution, leveraging existing pre-trained foundation models, including MiniLM text embedding models [19], DINOv2, an image feature extraction model [6], and Talk2DINO, a mapping model that translates text embeddings into the image feature space [10]. This ensures reproducibility, simplicity in application, and controllable computational cost.

The configurable storage option between using a vector database and knowledge graph storage enables pipeline choice depending on actual usage needs. Vector database and dense retrieval provide fast and scalable storage and retrieval, while knowledge graph storage may offer better handling for long-context relations; though our experiments did not confirm this advantage.

Generation grounding on both textual and visual modalities is supported, ensuring accuracy with the most retrieved evidence.

2.1.3 Design Tradeoffs

We utilize DINOv2 [6] as our visual feature extractor, for its superior fine-grained visual detail preservation from self-supervised learning. Although it does not provide a natural language interface, we utilize an additional layer Talk2DINO [10] to map text tokens into the same representation space, incurring slightly more computation.

As for cross-modal alignment, CLIP [5] inherently provides an alignment mapping between open-vocabulary natural language and visual content. However, surveys on existing multimodal RAG methods which mainly make use of CLIP for multimodal alignment reveal that CLIP-based vision encoding does not capture fine-grained visual details in images, or demonstrates a bias towards text instead of multimodal elements [2, 3]. We hence use Talk2DINO [10] to align text queries with DINOv2’s fine-grained visual features.

During retrieval, we generate a zero-shot answer to the input query to assist in retrieval. Since user queries may be brief, the hypothetical document often contains textual patterns

commonly found in the expected domain and answer, hence achieving higher recall in retrieval. This is known as the Hypothetical Document Embeddings (HyDE) technique [20] (more details in Section 2.2.5 Retrieval). Although hallucinated details may be introduced and propagated to the final answer, we adopt this method as the final answer remains grounded on real retrieved documents.

2.1.4 Design Validation

We validate design decisions using unit tests and integration tests (detailed tests and results in Section 2.3 Testing). Unit tests examine the capabilities of independent components of our pipeline using mock generated data, to verify that our pipeline components operate on input documents as expected. Integration tests examine the entire pipeline end-to-end, to examine actual results on small datasets for easy verification that the pipeline gives good performance.

Benchmark comparisons (in Section 2.4 Evaluation) compare our full pipeline against methods developed in previous work. We examine our pipeline’s performance with the benchmarks Visual-RAG [14] and MMDocRAG [16].

2.2 Implementation

The implementation of this project involves:

2.2.1 Dataset Collection

This project aims to explore a generic RAG pipeline that supports multimedia to be used as context in multimodal LLMs. As such, we collect and utilize datasets provided by publicly available benchmarks, to ensure wide domain and topic coverage.

Visual-RAG Visual-RAG [14] is a benchmark that explicitly requires text-to-image retrieval to answer queries and visually grounded generation with retrieved visual evidence. Benchmark dataset is based on the iNaturalist 2021 [15] dataset containing 2.7M images from 10,000 different species. We utilize this benchmark to evaluate our pipeline’s retrieval accuracy based on very fine-grained visual details.

MMDocRAG MMDocRAG [16] features over 4,000 expert-annotated question-answer pairs that rely on multi-page or multi-document cross-modal evidence. As visual evidence is a core evaluation metric in this benchmark, we evaluate our pipeline on this benchmark to compare with other models or methods, and to identify limits and guide future improvements.

We use these benchmarks to evaluate both retrieval and end-to-end answer generation using the final answer. Since our prompts (listed in Appendix A Prompt Templates) instruct the MLLM to answer only based on retrieved context, the final answer also reflects retrieval accuracy in a unified evaluation score.

2.2.2 Document Preprocessing

Input documents involve text and inline images. Our pipeline separates text and images in input documents for preprocessing, and fuses them partially for easy reference between relevant items.

As industry-grade documents can contain long content of up to hundreds of pages, we split document text into processable sizes. Input documents are first split into text chunks of fixed length with pre-defined fixed overlap [21], by recursive splitting of text to avoid cutting text mid-word into different chunks. The length and overlap between chunks are configurable constants.

As for images, we extract them from the document alongside text surrounding the inline image. We perform captioning for inline images using the extracted context from source documents to support captioning, such that document context is preserved when processing images in later steps. From testing different approaches (see Section 2.3.2 Image Extraction and Captioning for details, results, and analysis), we find that using MLLMs for captioning images provides the best tradeoff between computation cost and accuracy.

Chunks of text, their metadata, and images with their metadata are passed on to the next step in the pipeline for embedding.

2.2.3 Embedding

Chunks of text obtained from the preprocessing step are then embedded using a text encoder that maps short sentences or paragraphs into a high-dimensional dense vector space. The MiniLM

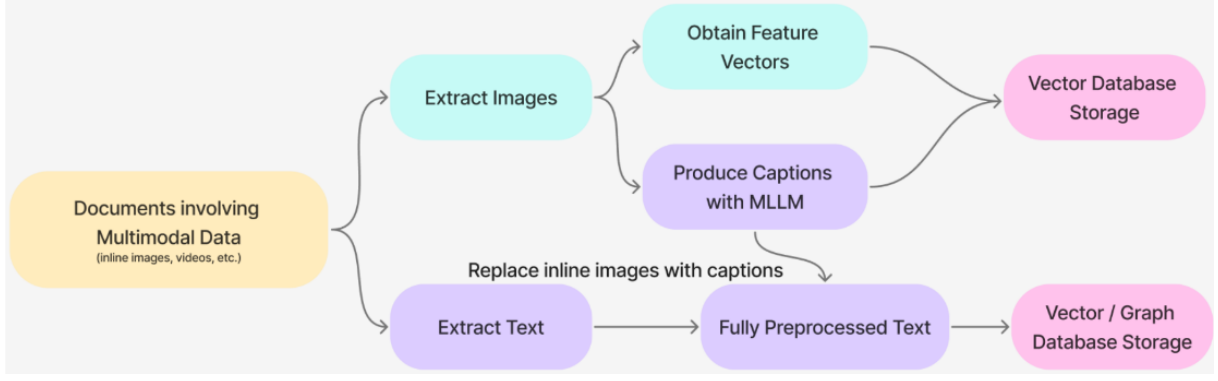


Figure 7: Flow of embedding input data and storage in knowledge base

models [19]¹ distilled from BERT [22] provide ideal performance in extracting semantic features from short texts into dense vectors. We use a MiniLM model to embed chunks of text and store the obtained dense vectors at preprocessing time, and encode queries into dense vectors in the same space to search in the knowledge base for similar text vectors.

Embedding multimedia data cannot be performed with the same method. Prior to embedding, each image is extracted for captioning conditioned on source document context surrounding the image using a multimodal LLM. Within the source document, the image is replaced with the obtained caption, to obtain a fully textual of the document for embedding with the above methods.

To obtain embeddings of images, we use a generic vision model that extracts visual features into dense feature vectors. DINOv2 [6] is a foundational vision model trained by self-supervision and gives feature vectors of images. We utilize DINOv2 in embedding images. Image embeddings are then placed in a vector database storage separate from the processed text embeddings, and similar or relevant items are linked to each other via metadata.

2.2.4 Storage

Chunks of processed data are embedded into text embedding vectors and stored in an external knowledge base. This is implemented using a vector database that provides storage and quick query functions.

¹The original paper gives details on training and the performance of the base model microsoft/MiniLM-L12-H384-uncased. In this project, we use a more powerful model, sentence-transformers/all-MiniLM-L6-v2, which is a fine-tuned version of the base model with a contrastive learning objective on a large dataset of sentences pairs.

Image feature vectors are stored in a separate vector database; with the difference being that a multi-vector database is used. Image vectors and caption embeddings are connected to each other during storage, to allow text queries to also be used during the retrieval process.

An alternate storage framework using knowledge graphs has also been implemented, following [17]. Using graph storage is an option to be chosen in the configurations. Entities are extracted from source document texts and form nodes in the graph, and relationships between entities form graph edges. This is extended to be connected with the image vector database through entity metadata. Communities of nodes in the knowledge graph are detected using Louvain’s algorithm, and community summaries are generated for each community using an LLM.

2.2.5 Retrieval

User queries to an MLLM may involve text only, or both text and reference images or documents. Processed documents stored into the knowledge base may contain references or correct answers to the query, hence relevant items are retrieved from the database to augment and ground generation to retrieved evidence.

Prior to retrieval, we offer an option to enhance the user’s raw query using the Hypothetical Document Embeddings (HyDE) technique [20]. A zero-shot prompt is first generated as a hypothetical document without any context and purely based on the query, and is then used for retrieval from the external vector database. While the generated hypothetical document may be highly incorrect and contain numerous hallucinations, the generated document contains the semantic nuances and patterns of texts on the same topic or domain. This text is then embedded using the same text encoder used in processing external documents, and the obtained embeddings are then used for text-to-text retrieval. Since our tests show no significant advantage in using HyDE (see Section 2.3.5 Ablation Tests for results and analysis), we leave this as a configurable option.

User queries may involve both text and images. While image-to-image can simply be done through passing query images through the same DINOv2 model to obtain a feature vector to perform dense retrieval, cross-modal retrieval by using text to retrieve images requires further

steps. CLIP [5] unifies feature vector representations of text and images, yet it does not provide accurate visual features existing in models. Talk2DINO [10] provides a projection layer from the CLIP embedding space to the DINOv2 embedding space, bridging the gap between text features and visual features vector spaces. Talk2DINO was trained by freezing both CLIP and DINOv2 backbones and achieves state-of-the-art results, hence we utilize this approach in performing cross-modal operations.

Queries for text-to-image cross-modal retrieval may be long to contain more expected content in images. The original CLIP model, used in Talk2DINO to encode text, suffers from a context size limit of 77 tokens, which is largely limited in this detailed retrieval use case if detailed visual elements are concerned. We hence utilize Long-CLIP [13] as the text encoder in the cross-modal retrieval step, to support longer queries that involve more detailed descriptions of expected visual features.

Following from the embedding and storage methods used, cosine similarity between vector embeddings is the primary retrieval criteria. Retrieval is performed through the following components:

- **Text-to-text search:** Query text to retrieve similar text chunks from the textual vector database using text embedding cosine similarity.
- **Image-to-image search:** Images in the query, if any, are passed through the image feature extraction model to obtain a feature vector. This feature vector is then used to search for similar images in the image multi-vector database, using cosine similarity between image feature vectors.
- **Text-to-image search:** Query text is embedded into the DINOv2 embedding space using Talk2DINO [10], a model that aligns text embeddings into the visual feature vector space of DINOv2 [6]. Similarly, images are retrieved using cosine similarity between the aligned text embeddings and image feature vectors.

When the graph-based storage method option is chosen, text-to-text search is implemented differently. The user query and community summaries are embedded with the same MiniLM text embedding model. We compare the relevance of a community by the cosine similarity

between embeddings of the user query and the community summary. Since we consider the top k most relevant items, we retrieve the k communities with the highest cosine similarity scores with the user query. We then aggregate the summaries of these communities as the retrieval result and provide an answer to the query.

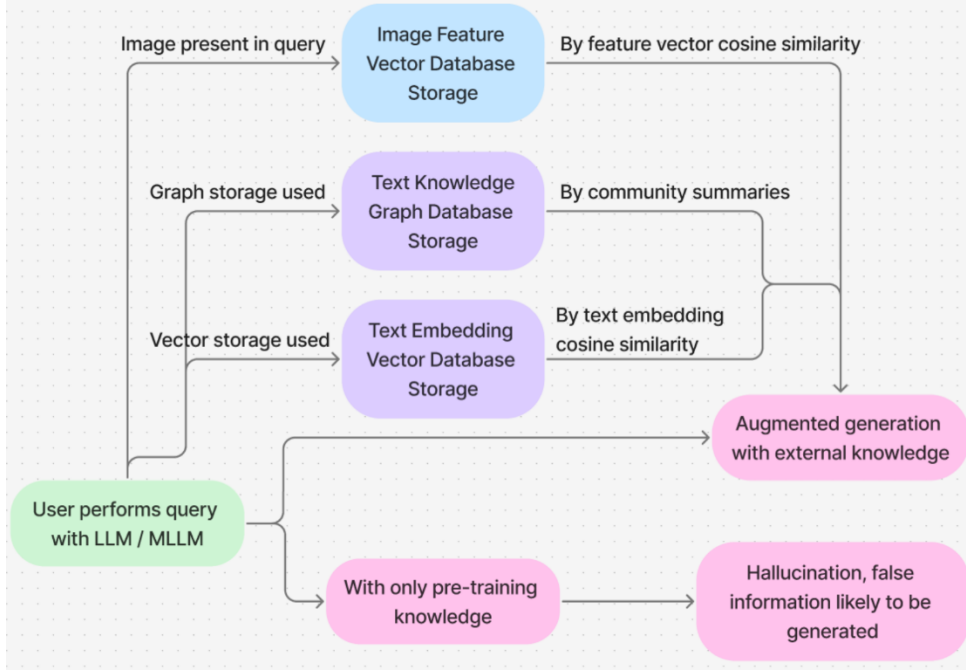


Figure 8: A high level overview of the entire project and implementation

2.2.6 Formal Task Definition

Given a set of n documents $\mathcal{D} = \{d^{(i)}\}_{i=1}^n$, where each document $d^{(i)}$ is composed of text $t^{(i)}$ and a set of p images $\mathcal{I}^{(i)} = \{m_j^{(i)}\}_{j=1}^p$, the following steps are carried out.

Image Captioning Each image extracted from the document is first captioned using an MLLM using text surrounding the image $t_{m_j^{(i)}}^{(i)}$ as context.

$$\mathcal{P}^{(i)} = \{p_j^{(i)} \mid p_j^{(i)} = \text{MLLM}(x \parallel t_{m_j^{(i)}}^{(i)} \parallel m_j^{(i)})\} \quad (1)$$

where x is the prompt instructing the MLLM to provide captions, and $\parallel$ indicating concatenation into the full prompt. The captions $\mathcal{P}^{(i)}$ are then inserted back into the text in the corresponding image position before further textual processing.

Chunking The text in the document is first split into chunks of fixed size L with a fixed overlap of O characters between consecutive chunks. Let $t_{\text{combined}}^{(i)} = t^{(i)} \cup \mathcal{P}^{(i)}$ be the full text with inserted image captions. The chunking process creates overlapping chunks indexed from $j = 0$ to m , where:

$$c_j^{(i)} = t_{\text{combined}}^{(i)}[s_j : e_j] \quad (2)$$

with start and end positions:

$$\begin{aligned} s_j &= j \cdot (L - O) \\ e_j &= s_j + L \end{aligned} \quad (3)$$

where L is the chunk size and O is the overlap length. Adjacent chunks $c_j^{(i)}$ and $c_{j+1}^{(i)}$ share O characters of overlap to preserve context. The total number of chunks is $m = \lceil \frac{|t_{\text{combined}}^{(i)}| - L}{L - O} \rceil + 1$.

Embedding and Storage Each chunk is embedded into a vector $\mathbf{v}_j^{(i)}$ using a neural text embedding model $\mathcal{N}_{\text{text}}$.

$$\mathbf{v}_j^{(i)} = \mathcal{N}_{\text{text}}(c_j^{(i)}) \quad (4)$$

These vectors are stored into an external vector database $\mathcal{S}$.

Alternatively in the graph-based storage option, first use an LLM to extract entities and relationships between entities from the source text. Entities form a set of nodes N in the graph, and relationships form edges E in the graph. The graph is formed as

$$\mathcal{G} = (N, E) \quad (5)$$

Using Louvain’s Algorithm, we locate communities $C = \{C_1, C_2, \dots\}$ in the graph and provide a community summary for each community using an LLM.

$$C_{\text{summaries}} = \{\text{LLM}(C_j)\} \quad (6)$$

Images are embedded using an image feature extraction model, DINOv2, to obtain image

embeddings.

$$\mathbf{g}_j^{(i)} = \text{DINOv2}(m_j^{(i)}) \quad (7)$$

Image embeddings and their caption embeddings will then be stored into a multi-vector database $\mathcal{S}_{\text{images}}$ as tuples:

$$\left(\mathbf{g}_j^{(i)}, p_j^{(i)} \right) \quad (8)$$

Retrieval Retrieval is performed using a user query $\mathcal{Q} = (q_{\text{text}}, \mathcal{Q}_{\text{images}})$, which is a tuple of query text and a set of query images. We consider top k items retrieved, and $\text{sim}(\cdot, \cdot)$ as the similarity metric.

- Using the query text, first retrieve k text chunks that have the highest embedding similarity with the query.

$$\mathcal{C}^R = \{ \mathbf{v}_j^{(i)} \in \mathcal{S} \mid j \in \text{top-}k \text{ indices}(\text{sim}(\mathbf{v}_j^{(i)}, \mathcal{N}_{\text{text}}(q_{\text{text}}))) \} \quad (9)$$

- If there are images in the query, i.e. $\mathcal{Q}_{\text{images}} \neq \emptyset$, obtain a feature representation for each image q_{images} and retrieve similar images.

$$\mathcal{I}^R = \bigcup_{q_{\text{images}} \in \mathcal{Q}_{\text{images}}} \{ \mathbf{g}_j^{(i)} \in \mathcal{S}_{\text{images}} \mid j \in \text{top-}k \text{ indices}(\text{sim}(\mathbf{g}_j^{(i)}, \text{DINOv2}(q_{\text{images}}))) \} \quad (10)$$

- Finally, the cross-modal retrieval step by embedding the query text in DINOv2 feature vector space and retrieve similar images.

$$\mathcal{I}^R = \mathcal{I}^R \cup \{ \mathbf{g}_j^{(i)} \in \mathcal{S}_{\text{images}} \mid j \in \text{top-}k \text{ indices}(\text{sim}(\mathbf{g}_j^{(i)}, \text{Talk2DINO}(q_{\text{text}}))) \} \quad (11)$$

Under the graph storage option, items are retrieved by scoring community summaries against the query. First, we embed the query text and each community summary:

$$\mathbf{q}_{\text{embed}} = \mathcal{N}_{\text{text}}(q_{\text{text}}) \quad (12)$$

$$\mathbf{s}_j = \mathcal{N}_{\text{text}}(C_{\text{summaries}}[j]) \quad \forall C_j \in C \quad (13)$$

For each community, we compute a relevance score indicating the usefulness of that community summary in answering the query:

$$s_j = \text{sim}(\mathbf{q}_{\text{embed}}, \mathbf{s}_j) \quad \forall j \quad (14)$$

Communities are ranked by their relevance scores in descending order, and the top k communities are selected for retrieval:

$$C^R = \{C_j \mid j \in \arg \max_{\text{top } k}(s_1, s_2, \dots, s_{|C|})\} \quad (15)$$

where $\arg \max_{\text{top } k}$ returns the indices of the k largest scores. The selected community summaries are then aggregated to provide context for answer generation.

Generation Output generation is performed with an MLLM by conditioning generation on retrieved text and images. The final answer a is generated by concatenating retrieved text, query text, retrieved images, and query images.

In dense vector retrieval mode, this is formulated as

$$a = \text{MLLM}(C^R \parallel q_{\text{text}} \parallel \mathcal{Q}_{\text{images}} \parallel \mathcal{I}^R) \quad (16)$$

For graph retrieval mode, $\mathcal{C}_{\text{graph}}^R = \{(C_{\text{summaries}}[j], E_j^{\text{key}}) \mid j \in \arg \max_{\text{top } k}(s_j)\}$ which is the set of top- k community summaries with their associated key entities E_j^{key} . We aggregate all community summaries

$$\mathcal{C}_{\text{aggregated}}^R = C_{\text{summaries}}[1] \parallel C_{\text{summaries}}[2] \parallel \dots \parallel C_{\text{summaries}}[k] \quad (17)$$

where $\parallel$ indicates a concatenation operation. Generation is formulated as

$$a = \text{MLLM}(\mathcal{C}_{\text{aggregated}}^R \parallel q_{\text{text}} \parallel \mathcal{Q}_{\text{images}} \parallel \mathcal{I}^R) \quad (18)$$

When grounded with retrieved context and evidence from related source documents, gen-

eration variance is greatly reduced; and timely or inaccessible knowledge outside of model knowledge can be utilized. This avoids incorrect or out-of-context answers from being generated.

A summary of all the notations used is available at Appendix B Summary of Mathematical Notations.

2.3 Testing

We perform unit tests on individual components to ensure sanity checks are passed. We also implement integration tests to verify that components and the end-to-end pipeline interact with external services (e.g. MLLM API calls) as expected. In integration tests with actual MLLM calls, we use GPT-4o-mini for tests.

2.3.1 Unit Tests

Unit tests are written and conducted with `pytest` version 8.3.4. The list of components tested independently with unit tests is as follows.

Component	Tests	Example Test Case
Document Processor	Load document, text chunking, directory processing (6 tests)	Process a directory of documents, expect non-zero number of chunks, expect chunks overlap, expect number of chunks and metadata are equal
Vector Database	Add documents, search documents (4 tests)	Insert known string and vector, expect same string to be returned when querying with the vector
Image Storage	Store image, retrieve image data URL (3 tests)	Create random image bytes and the corresponding image data URL, store image bytes, expect retrieved data URL to be correct

Component	Tests	Example Test Case
Text Embedding	Test encode single string and list of strings (2 tests)	Encode given strings, expect correct dimensionality, expect correct number of embedding vectors
Multimodal Embedding	Conversion between data URL and PIL image, embed image, embed query, consistency (13 tests)	Encode given images, expect correct dimensionality
Graph Storage	Entity and relationship extraction, build community (7 tests)	Given known corpus, create a graph from entities and relationships extracted, expect graph with non-zero number of nodes and edges
Graph Retriever	Retrieve entity, search global question (7 tests)	Given a graph with given entities and relationships, search and retrieve items, expect correct items to be retrieved
Main class	Methods in the main entrypoint class (10 tests)	Initialize system, ingest documents, query
End-to-end pipeline	Interaction between components (10 tests)	Run through entire pipeline from initialization to query

We also test edge cases. Adding empty documents for processing in the vector database or graph storage are tested to be handled correctly without error. Conversions between image data URLs and PIL images with invalid data URLs are also examined and shown to fail gracefully as expected.

Among all 62 unit tests, the passing rate is 100%.

2.3.2 Image Extraction and Captioning

We select 4 documents² from the Internet to perform integration tests. These documents contain inline images covering specific mathematics figures, conceptual images in advertisements, informational images with text, and images with advanced concepts that require document context to explain.

Preprocessing by captioning inline images in documents is required in the pipeline. We perform integration tests on captioning in-document images with various methods, to test the image extraction process, and examine the quality and coherence of captions generated for each image, as well as the time used.

CLIP In this trial, captions were generated following the model implementation of ClipCap [23]. As we observed that incorrect context was given in the captions, we refined the captions with an LLM conditioning on document context surrounding the image.

We note that the refined captions are highly influenced by the CLIP-based model caption, and hence out-of-context captions were frequently observed.

Multimodal LLM We perform another test by using a MLLM (GPT-4o-mini) to produce captions for images extracted from documents. The prompt involved instructions on captioning, the image, and context from the source document surrounding the image.

A significant improvement in accuracy was observed, at the cost of around 40% longer processing time with larger variability, as compared to the previous method. However, we choose this method for integration with the pipeline.

²The documents are linked as follows:

[https://math.libretexts.org/Bookshelves/Calculus/Calculus_\(OpenStax\)/05%3A_Integration/5.01%3A_Approximating_Areas](https://math.libretexts.org/Bookshelves/Calculus/Calculus_(OpenStax)/05%3A_Integration/5.01%3A_Approximating_Areas)

<https://www.apple.com/hk/en/newsroom/2025/11/apple-intelligence-expands-to-new-languages-including-traditional-chinese/>

<https://cse.hkust.edu.hk/pg/research/projects/saikit/underwater-slam/>

Note: One of the documents was an email from HKUST to university members and is not publicly available.



Figure 9: An example image passed through the captioning test, extracted from an email issued by HKUST to university members about transport arrangements.

Caption from the CLIP-based model: ‘the bus is a popular tourist attraction’ (score: 0.25)

Enhanced caption: ‘The bus is an important transport option for students, offering a direct service to the campus on the 91B route.’ (score: 1)

Test Results and Statistics We employed the following scoring rules in assessing the accuracy of captions generated.

- 0: Completely out-of-context.
- 0.25: Partially correct, but brings in incorrect context.
- 0.5: Partially correct; does not connect the correct part with the incorrect part as context.
- 0.75: Largely correct, little incorrect context is seen.
- 1: Completely correct.

We annotated scores for each image manually by referring to the source document contents.

Table 2: Accuracy statistics of different methods

	CLIP-based Model	CLIP-based (Enhanced)	Multimodal LLM
Document 1			
(specific mathematical figures)	0.0313	0.5156	0.9844
Document 2			
(conceptual images)	0.0208	0.4792	0.9792
Document 3			
(informational images with text)	0.1875	0.6875	0.9375
Document 4			
(advanced concepts)	0.25	0.875	1.0
Overall	0.0588	0.5441	0.9779

Table 3: Statistics on the amount of time used per image

	CLIP-based Model (Enhanced)	Multimodal LLM
Mean (s)	2.7507	3.8861
Variance (s ²)	1.0171	2.9140
Maximum (s)	7.2277	10.1240
Minimum (s)	1.7325	1.7750

2.3.3 Retrieval

We use the same corpus of 4 documents from the captioning integrated tests for retrieval tests. One of the documents from the corpus contains an introduction to a feature named “Genmoji”, with an example “rainbow cactus” given. We specifically set up a query that targets this section of the document:

What is Genmoji like and how can I use it?

Can you give me an example?

The top text chunk among the retrieval results is:

Retrieved document 1:

or combine emoji and text to create something entirely new. With images from their photo library, users can take Genmoji even further by creating one that is inspired by a friend or family member. And similar to Image Playground, users can also change expressions or modify personal attributes to match their contact’s latest look. Users can create their own unique emoji with Genmoji, making conversations with family and friends more fun and playful.

Take Notes to the Next Level with Image Wand

With Image Wand in the tool palette, users can quickly create images in the Notes app using the written or visual context already captured within a note.

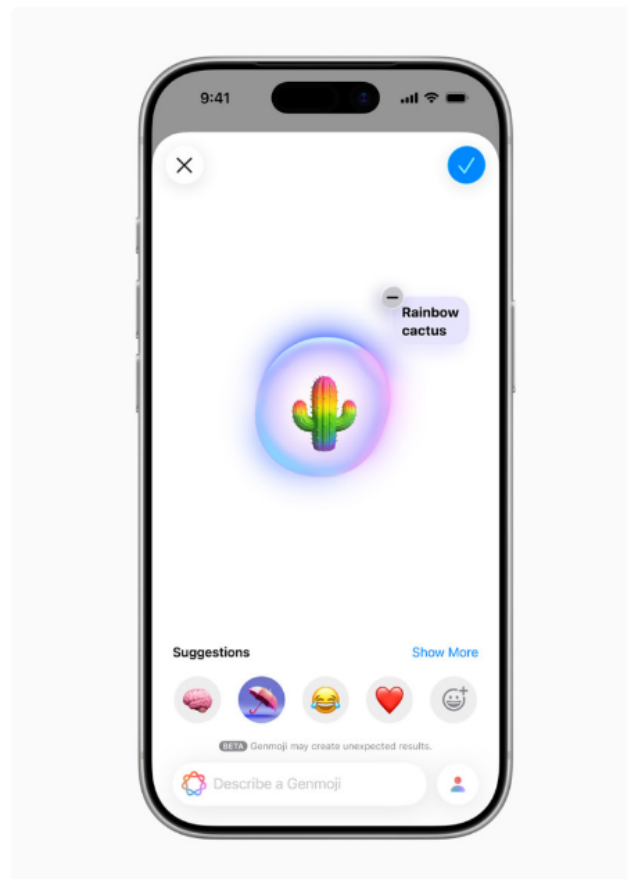
This is correctly retrieved as the chunk mentions use cases of the “Genmoji” function.

With multimodal retrieval enabled, we find that the image depicting a “rainbow cactus” is also correctly retrieved and used as visual context in generating the answer.

Create Genmoji to Fit Any Moment

Users can create their own unique emoji with the power of Apple Intelligence, making conversations with family and friends more fun and playful. Users can type a description to create an emoji. They can also mix together multiple emoji, or combine emoji and text to create something entirely new. With images from their photo library, users can take Genmoji even further by creating one that is inspired by a friend or family member. And similar to Image Playground, users can also change expressions or modify personal attributes to match their contact's

latest look.



Users can create their own unique emoji with Genmoji, making conversations with family and friends more fun and playful. ↓

Figure 10: Part of a document from the test corpus

2.3.4 Grounding Generation Output

We examine the generation results of the previous example on “Genmoji”. One final answer when multimodal mode is enabled is

Genmoji is a fun feature that allows users to create personalized emojis for their conversations. You can design your own unique emoji by typing in a description or mixing and matching existing emojis. This feature adds a playful touch to your messaging, making it more engaging with family and friends.

For example, you can create a "rainbow cactus" emoji by describing it, and then you can further customize it by adjusting colors or combining it with other elements. You can also use photos from your library to inspire your designs, allowing for even more creative possibilities.

Overall, Genmoji is all about enhancing communication through visually appealing and personalized emojis that reflect your style and the context of your conversations.

This answer correctly utilizes retrieved context to generate a comprehensive answer on the functionality of “Genmoji”, and provides a correct example, which was also present in the retrieved context.

We also check the results for when text-only retrieval is allowed, and for zero-shot generation. The same text chunks are retrieved for text-only retrieval, resulting in the following final answer:

Genmoji is a fun and creative feature that allows you to create your own unique emojis. You can make conversations with family and friends more playful by using Genmoji to

craft emojis that reflect your personal style or express your feelings.

To use Genmoji, you can type a description of what you want the emoji to look like, mix together multiple existing emojis, or even combine emoji and text to create something entirely new. Additionally, you can use images from your photo library to inspire your creations, such as designing an emoji that resembles a friend or family member.

For example, if you wanted to create an emoji inspired by your friend who loves cats, you might start by mixing a cat emoji with a party hat emoji to reflect their fun personality. You could describe it in your own words, like "A cat with a colorful party hat," and Genmoji would help you bring that idea to life!

This answer is also correct as the retrieved text chunks contain fully the information to answer this question, and since visual evidence does not play a very big role in the second subquestion asking for examples, which is a more open-ended and creative question. We can also see that the example given here does not follow the source document image example "rainbow cactus", as the model has no access to such visual context during generation.

As for zero-shot generation, the generated answer is

Genmoji is a creative tool that allows users to generate personalized emojis and stickers based on their preferences or input. It's often used for fun and self-expression, enabling users to create custom emoji that represent their personality or emotions in a more tailored way than standard emojis.

How to Use Genmoji:

1. ****Access the Platform****: Visit the Genmoji website or app where the service is available.
2. ****Select Options****: You may have options to choose various features such as facial expressions, accessories, hairstyles, and backgrounds.
3. ****Customize Your Emoji****: Use the provided tools to customize your emoji to your liking. You can usually adjust colors, add features, or apply different styles.
4. ****Save or Share****: After creating your emoji, you can save it to your device or share it on social media platforms.

Example:

Let's say you're feeling happy and want an emoji that reflects that:

1. ****Choose a Base Expression****: Start with a smiling face.
2. ****Add Accessories****: Put on some sunglasses and a party hat.
3. ****Customize Colors****: Change the skin tone to match yours.

4. ****Finalize and Share****: Save this emoji to your phone and use it in messages to friends or on social media to express your cheerful mood!

By using Genmoji, you can create unique representations of yourself or your feelings, making communication more vibrant and personal.

We find multiple issues in this answer. For instance, the said “Genmoji” feature in the source document does not refer to a website application, yet this zero-shot generated answer confidently states that access to “Genmoji” is via a website.

We formulate another small corpus of 4 documents for more comprehensive testing on visual retrieval and grounding. Dataset questions are crafted using document content, targeting specific visual elements found in the document. More details on the dataset and corpus can be found in Appendix C Retrieval and Generation Integration Tests.

The results by running zero-shot, text-only retrieval, and multimodal retrieval on GPT-4o-mini are as follows. We provide tuples of scores representing (dense retrieval, graph retrieval) scores.

Value of k	Text-only	Multimodal
0 (Zero-shot)	27.3	–
1	36.4, 9.1	45.5 , 18.2
3	36.4, 18.2	45.5 , 45.5
5	45.5 , 18.2	27.3, 27.3
7	36.4, 9.1	54.6 , 9.1
10	45.5, 18.2	54.6 , 36.4
15	45.5 , 9.1	27.3, 18.2
20	45.5 , 9.1	36.4, 9.1

We see that the level of accuracy increases from zero-shot to text-only retrieval, then to multimodal retrieval.

2.3.5 Ablation Tests

CLIP vs DINOv2 Embeddings We perform ablation tests on the corpus of 4 documents used in Section 2.3.4 Grounding Generation Output for using CLIP embeddings in multimodal embedding and retrieval. To accomodate long-context strings, we use Long-CLIP [13] instead of the vanilla CLIP model.

Test results are as follows. Tuples of scores represent (dense retrieval, graph retrieval) scores.

Value of k	Text-only	Multimodal (DINOv2-Talk2DINO)	Multimodal (Long-CLIP)
0 (Zero-shot)	27.3	–	–
1	36.4, 9.1	45.5 , 18.2	36.4, 18.2
3	36.4, 18.2	45.5 , 45.5	45.5 , 18.2
5	45.5 , 18.2	27.3, 27.3	45.5 , 27.3
7	36.4, 9.1	54.6 , 9.1	27.3, 9.1
10	45.5, 18.2	54.6 , 36.4	27.3, 36.4
15	45.5 , 9.1	27.3, 18.2	36.4, 18.2
20	45.5 , 9.1	36.4, 9.1	36.4, 18.2

The DINOv2-Talk2DINO method achieves higher accuracy for $k=1, 3, 7, 10$ as compared to using CLIP (Long-CLIP) for multimodal embeddings, and we also note that the highest accuracy among all settings is under the DINOv2-Talk2DINO pipeline. For larger values of k such as $k=15$ and 20, text-only retrieval outperforms multimodal retrieval. This suggests that although visual retrieval provides fine-grained visual information for smaller retrieval sets, larger batches may introduce noise from irrelevant retrieved items, confusing the MLLM and resulting in lower accuracy. Long-CLIP multimodal embedding retrieval underperforms DINOv2 in most settings, as the unified representation space for text and images loses fine-grained details for semantic alignment.

These results validate our design decision to use separate representations and dual modality retrieval, especially when the number of retrieved items remains moderate, such as for $k \leq 15$.

HyDE We perform experiments on comparing accuracy results between using HyDE or raw query embeddings for dense retrieval. Our results are as follows, where scores are in tuples (using HyDE, using raw query).

Value of k	Text-only	Multimodal
0 (Zero-shot)	27.3	–
1	36.4, 27.3	45.5, 54.6
3	36.4, 54.6	45.5, 36.4
5	45.5 , 36.4	27.3, 45.5
7	36.4, 45.5	54.6 , 45.5
10	45.5, 36.4	54.6 , 36.4
15	45.5 , 36.4	27.3, 36.4
20	45.5 , 36.4	36.4, 36.4

Among the 14 tested settings, retrieval with HyDE won in 8, while retrieval using the raw query won in 5, and there was one tie. No clear trend with respect to k was observed. For instance, using the raw query for retrieval gave better performance at $k=1$ in multimodal retrieval, but worse in text-only retrieval. Given these results that lack a clear conclusion for the better method, and an additional computation cost in generating the hypothetical document under HyDE, we leave both methods as a configurable option for flexibility.

Note on Dataset We also note that this test dataset of 4 documents and 11 queries is a small dataset, and may introduce bias in test results. However, this small-scale testing allows us to perform deeper investigations on failed cases.

A more comprehensive evaluation using research benchmarks can be found in Section 2.4 Evaluation.

2.4 Evaluation

To evaluate overall pipeline performance, we run our pipeline through benchmarks. We examine retrieval accuracy and end-to-end answer generation on the benchmarks Visual-RAG and MMDocRAG.

2.4.1 Benchmarks

We present results of running our system on various benchmarks as compared to the original papers by using GPT-4o-mini³ and Qwen2.5VL with our pipeline. Accuracy scores shown below are obtained using the LLM-as-judge technique [24]. The prompt used in LLM-as-judge evaluation is listed in Appendix A Prompt Templates.

Visual-RAG Items in brackets represent results from the original paper. Visual-RAG specifically targets visual elements, hence there is no text corpus; only images are indexed. Top- k retrieval refers to retrieval of the top k most similar images to the query based on our metric. Note that the original paper does not provide results for GPT-4o-mini. We cite scores for GPT-4o from the original paper as a reference.

Model	GPT-4o-mini (GPT-4o)	Qwen2.5VL
Zero-shot (no image)	43.05 (49.33)	56.68 (34.84)
Top- k , $k=1$	7.49 (22.65)	37.43 (37.96)
Top- k , $k=3$	11.76 (38.50)	33.42 (39.44)
Top- k , $k=5$	15.24 (43.45)	39.57 (39.84)
Top- k , $k=7$	14.44 (46.52)	40.91 (38.99)
Top- k , $k=10$	17.11 (46.93)	42.25 (42.91)
Top- k , $k=15$	19.25 (47.33)	46.79 (42.78)
Top- k , $k=20$	17.91 (50.00)	43.58 (43.44)

³Provided by Microsoft Azure via HKUST GenAI Platform.

Due to limits on compute resources and time, we do not use the full image dataset of 2.7M images from the iNaturalist 2021 Competition training set [15], as in the original paper; instead we simulate the experiment with the test set of 100,000 images. We find that Qwen2.5VL gives comparable results to the original paper although the number of ground truth images for each query is largely reduced. We also notice that scores for GPT-4o-mini lag behind GPT-4o.

We perform further analysis on retrieval results. We report numbers for $k=7$ and 15 additionally. Results compared to the original paper are as follows:

Ours	@1	@5	@7	@10	@15	@20
Recall	3.40	18.37	22.75	25.29	28.13	30.43
NDCG	33.96	35.58	31.44	22.16	21.93	22.07
Hit	33.96	70.86	70.86	70.86	76.47	76.47
Hit Count	0.34	1.84	2.28	2.53	2.81	3.04
CLIP	@1	@5	@7	@10	@15	@20
Recall	2.81	10.26	–	16.70	–	25.54
NDCG	24.33	25.97	–	30.98	–	39.21
Hit	24.33	53.74	–	67.65	–	77.54
Hit Count	0.24	1.01	–	1.84	–	3.18
BGE	@1	@5	@7	@10	@15	@20
Recall	3.63	12.43	–	18.81	–	28.64
NDCG	26.74	29.15	–	33.4	–	41.83
Hit	26.74	58.56	–	68.98	–	79.14
Hit Count	0.27	1.17	–	2.04	–	3.43
VLM2Vec	@1	@5	@7	@10	@15	@20
Recall	2.91	10.01	–	16.03	–	26.37
NDCG	24.87	26.33	–	30.09	–	39.13
Hit	24.87	51.87	–	62.83	–	78.34
Hit Count	0.25	1.10	–	1.91	–	3.33

The above results are analysis results of retrieval logs from the end-to-end generation experiment. We use the same evaluation metrics on retrieval results as the original paper, including the following:

- Recall @ k : the fraction of ground truth images among the top k retrieved images
- Normalized Discounted Cumulative Gain (NDCG) @ k : reflects ranking quality by rewarding correct images that appear higher in the ranking, by discounting relevance logarithmically with position
- Hit @ k : the probability of at least one clue image among the top k images
- Hit Count @ k : the number of clue images found in the top k retrieved images

We note that experiment conditions slightly differ from the original paper. In retrieval evaluation, the authors of the original paper select 200 to 300 images for retrieval per question; while we perform retrieval among all 100,000 images in the test image dataset used in our experiments, making our experiment conditions more difficult for the retriever.

The above results show that our DINOv2-Talk2DINO pipeline provides competitive recall compared to methods tested in the paper. However, due to a smaller ground truth set, where each species has only 10 ground truth images, and a much larger corpus, our NDCG values decline for $k > 5$.

Our results suggest that using DINOv2 representations and aligning textual queries with them using Talk2DINO may allow some minor fine-grained visual details to be considered in the retrieval process.

MMDocRAG Items in brackets represent output factuality score results from the original paper. We provide scores for zero-shot and $k=5, 7$ and 10 additionally.

Model	GPT-4o-mini	Qwen2.5VL ⁴
Pure-text input sequence to MLLM		
Zero-shot (no retrieval)	49.9 (–)	69.2 (–)
5 quotes ⁵ (Top- k , $k=5$)	50.4 (–)	59.6 (–)
7 quotes ⁵ (Top- k , $k=7$)	51.1 (–)	59.8 (–)
10 quotes ⁵ (Top- k , $k=10$)	54.0 (–)	61.0 (–)
15 quotes ⁵ (Top- k , $k=15$)	56.9 (70.0)	61.8 (61.2)
20 quotes ⁵ (Top- k , $k=20$)	57.4 (69.8)	62.0 (61.4)
Multimodal input sequence to MLLM		
5 quotes ⁵ (Top- k , $k=5$)	52.7 (–)	65.8 (–)
7 quotes ⁵ (Top- k , $k=7$)	55.6 (–)	66.4 (–)
10 quotes ⁵ (Top- k , $k=10$)	55.6 (–)	67.0 (–)
15 quotes ⁵ (Top- k , $k=15$)	57.5 (66.7)	66.4 (63.2)
20 quotes ⁵ (Top- k , $k=20$)	55.5 (64.6)	65.8 (62.6)

We note that our results on GPT-4o-mini lag behind the original paper; yet results on Qwen2.5VL outperforms the original paper in most settings. This phenomenon may be due to performance differences between GPT-4o-mini offered by Microsoft Azure⁶ and the official GPT-4o-mini from OpenAI, or model versioning differences. Although generation with retrieval only provides marginal gains over zero-shot generation for GPT-4o-mini, the factuality scores increase as more items are retrieved and used as context. This trend indicates that more provided context aids generation in a correct direction.

The results for Qwen2.5VL further validate that our DINOv2-Talk2DINO pipeline provides

⁴For pure-text evaluations, the original paper provides results from Qwen2.5-7B-Inst. We compare with such results.

⁵‘Quotes’ refers to the number of retrieved passages.

⁶We use GPT-4o-mini offered by Microsoft Azure through HKUST GenAI Platform.

the MLLM with useful multimodal context extracted from document images, such as graphs, tables, and explanatory images.

Discussion on Zero-Shot Generation We find that zero-shot generation for both Visual-RAG and MMDocRAG with Qwen gives higher accuracy than outputs under retrieval; whereas scores for GPT-4o-mini follow the expected trend and increase as more items are being retrieved. We hypothesize that post-training with Supervised Fine-Tuning (SFT) on Qwen2.5 to achieve long-context generation and logical reasoning abilities [25] causes this phenomenon. Longer zero-shot generation output has a higher chance of hitting related factual points presented in the ground truth answers, leading to the LLM judge awarding more points to zero-shot answers. Generation with retrieval does not benefit from these model characteristics as we prompt for generation to be strictly based on retrieved context.

2.4.2 Overall Evaluation

Our evaluation of the overall pipeline with the following criteria is as follows:

1. Is the collected dataset large enough to evaluate the pipeline thoroughly?

We mainly use MMDocRAG and Visual-RAG as evaluation benchmarks. MMDocRAG contains 222 long documents and 4,055 expert-annotated question-answer pairs, while Visual-RAG utilizes a dataset of 2.7M images and 374 queries that require retrieval based on fine-grained visual details. This is sufficient to evaluate our pipeline’s performance.

2. Does the designed embedding strategy retain sufficient necessary information from the raw multimodal data?

We find that retrieval accuracy is high enough to ground generation in a correct direction, with multimodal data and textual data both used. This can be seen in our tests, where for a moderate number of multimodal retrieved items (such as $k=7, 10$), multimodal retrieval gives better accuracy than text-only retrieval. Benchmarking results with MMDocRAG also reveal the same. This suggests that the encoding method provides meaningful embeddings that can be retrieved correctly for use.

3. Is the storage mechanism efficient enough to provide quick retrieval upon actual use with a MLLM?

We observe that the vector database storage option allows retrieval of documents very rapidly in seconds, which is ideal performance for retrieval and passing to a MLLM for generation. The knowledge graph-based storage option takes slightly longer to retrieve items due to embedding community summaries on the spot; yet theoretically this option preserves long-form context better. Our test results, however, do not support this. The choice of storage mechanism lies within the actual use case of this system, and we leave the choice as a configurable option in our pipeline.

4. How relevant is the retrieved information when compared to the input query?

As observed in our tests and experiments, we find that retrieval using the HyDE technique or the raw query both allow effective retrieval of relevant text chunks and images. Our prompts also instruct the MLLM to strictly generate answers only based on retrieved items. Answer accuracy is an indicator for retrieval effectiveness in our tests. We note that retrieval increases accuracy compared to zero-shot baselines in all tests and benchmarks, hence we conclude that retrieved items can successfully ground generation and can largely reduce hallucination.

5. How helpful is the system in reducing hallucinations or factual errors when dealing with multimodal data?

Following from the previous question, relevant text chunks and images can successfully and correctly be retrieved prior to generation. The prompt for the final answer instructs the MLLM to appropriately utilize the provided retrieved context in answering the input query, and we find that grounded generation provides ideal performance in preventing hallucinations or factual errors. However, we also find that when too many retrieved multimodal items are retrieved and used as context, the MLLM may get confused and consequently produce incorrect answers. The optimal values of k from our empirical results are $k=7, 10$ or 15 .

6. How does this system compare to emerging research papers on similar topics?

Our benchmark results on MMDocRAG are on par with results from the original paper, and exceed original results in certain settings. We also find that our results on Qwen2.5VL are comparable to the original paper results in Visual-RAG, and further analysis on retrieval results shows that our method gives slightly better retrieval performance.

3 Discussion

3.1 Analysis of Findings

We analyze failure cases in integration tests and benchmarking for deeper insight into reasons for inaccurate answers.

Document Comprehension We find that document layout comprehension plays an important role in retrieval accuracy. Documents tested in benchmarks include real-world documents from a wide range of domains with various formats. For instance, presentation slides, academic papers with multi-column layouts, and informational leaflets or brochures that feature interleaved text in images occupying the entire page space. Our document processor extracts images and texts in reading order, hence possibly confusing multi-column document layouts for single-column layouts. While our pipeline is observed to successfully retrieve entire presentation slides to produce a correct answer, we also find that in some cases, retrieved raw text documents show content in the same line among multiple columns. Such noise may prevent correct generation of answers, causing failure cases.

Knowledge Graph Storage Our graph-based storage option extracts entities and relationships to form a knowledge graph. We then generate community summaries with an LLM for communities of entity nodes detected in the graph. Theoretically graph-based storage preserves long-form context more accurately; however in our tests we find that flattening source documents into summaries causes loss of specific details in source documents. This led to a gap between dense retrieval and graph community retrieval for augmenting generation. We hence default to using dense retrieval.

Generation Accuracy The level of generation accuracy increases from zero-shot to text-only retrieval, then to multimodal retrieval. Inline images may contain valuable fine-grained visual information useful in providing an answer to a query. By retrieving related images and passing them to the MLLM, we make full use of external timely knowledge to support generation grounding, increasing the accuracy of final answers. This is observed in our end-to-end tests,

where the highest accuracy score obtained from multimodal retrieval is 54.6%, and that of text-only retrieval only reaches 45.5%. Zero-shot generation gives 27.3% accuracy. Our findings verify our hypothesis that retrieving visual information helps in reducing hallucination and factual errors in answering queries.

3.2 Limitations

Despite our proposed pipeline’s strengths, we identify three major limitations: preprocessing efficiency, theoretical bounds of dense retrieval, and current MLLM visual perception capability issues.

Preprocessing The preprocessing step in our pipeline requires inline image extraction from documents and captioning. We adopt MLLM image captioning with source document context as this method gives the highest accuracy among all methods tested. While the accuracy is exceptional (reaching an average of 97.79% on our small test corpus), we find that processing time reaches 3.89 seconds per image. This creates a large indexing overhead for real-world uses, especially corporate-grade document batches that may contain thousands or millions of images. Using proprietary closed-source MLLMs via API requests might incur further networking or connection delays, while running inference on open-source MLLMs in local environments would demand much computation power and hence more powerful hardware.

Our proposed pipeline also does not support video processing. Similar work such as VideoRAG [18] already demonstrate the possibility of indexing long-form videos into a knowledge base (a knowledge graph in this specific implementation) and can be used for generation grounding. As videos are formed of continuous frames of images, it would be possible to adopt a similar approach to our pipeline in processing videos.

Dense Retrieval Bounds Our pipeline offers dense retrieval from a vector database or knowledge graph retrieval for the text-to-text retrieval step, yet dense retrieval in the DINOv2 vector space is the only method for text-to-image and image-to-image retrieval using the DINOv2 representation space. Although dense retrieval is extensively used in RAG applications [26, 1], recent research has revealed the theoretical limits of dense retrieval. [27] finds that dense retrieval faces

theoretical limits of retrieval effectiveness with regards to the total number of documents n , the number of documents to retrieve k , and the dimensionality of dense vectors used in indexing documents. In corporate-level applications of our pipeline, where the number of documents and images can reach large numbers, theoretical limits of dense retrieval may be reached. This may cause retrieval to be meaningless and provide no help in answering a query.

MLLM Perception This project is based on the assumption that retrieved images provided to a MLLM will further ground generation to visual evidence, hence conditioning generation on an extra dimension of ground truth information will allow the rate of hallucination and factual errors to be reduced. Current MLLMs demonstrate expert-level performance on domain-specific knowledge-intensive questions; but also show deficits in basic visual perception without textual descriptions as a fallback [28]. The paper finds that leading MLLMs lack behind adult humans on beyond-language visual tasks by up to 44.4% (best-performing MLLM 49.7%; adult humans 94.1%). This reflects a severe lack in MLLM visual perception capabilities when the item concerned cannot easily be translated or flattened into natural language.

Our pipeline assumes that fine-grained visual features will assist in answering vision-specific questions that cannot be covered by retrieved text. However, this assumption is largely limited by MLLM visual perception capabilities.

These limitations motivate the future directions described below.

3.3 Future Work

Better Preprocessing Efficiency Future work may explore more efficient methods for document preprocessing and inline image captioning to strike a better balance between the time taken and captioning accuracy. Current implementation in our pipeline incurs a linear time complexity in the number of documents and images, as processing is performed sequentially to avoid exceeding rate limits in API call requests. Parallel processing and calls, for instance, may be an improvement over the current method.

Foundational Models We utilize DINOv2, a foundational vision model, and Talk2DINO, a model that maps text representations into the DINOv2 visual representation space. These

models achieve state-of-the-art performance on various tasks and can be directly used in our pipeline. For vision-critical applications, the foundational models can be further fine-tuned using domain-specific datasets to further raise the capabilities of these models, then plugged into our pipeline to be used in multimodal RAG with a MLLM.

Retrieval Further retrieval methods can be employed in further work to further increase retrieval effectiveness. For example, emerging methods with agentic RAG [4] allow dynamic retrieval during generation to access additional visual evidence, as opposed to the current standard one-off pre-generation retrieval in our pipeline, which may be limited in evidence retrieval for question answering. Rerankers to re-sort the top- k retrieved items may ensure that the most relevant information is provided to the MLLM for grounding generation. Cross-modal rerankers could reorder the top- k retrieved images from DINOv2 embeddings based on fine-grained visual similarity to the query, improving precision at the cost of slightly added latency.

Perception Improvement Due to the limits of visual perceptive abilities of existing MLLM frameworks, researchers have proposed that a new representation space can be used for tokenizing and embedding text or image patches [28]. This may allow visual nuances to be better captured by newly developed models, such that generation can attend more to fine-grained visual elements, further increasing generation accuracy and reducing hallucination or factual errors.

Application Scenarios We apply our multimodal RAG on text-to-text and vision-to-text MLLMs only. In text-to-vision-enabled models, images and text can be used as queries for generating novel images [29]. Such tasks may make use of visual details and nuances in retrieved images to a much larger scale than text generation tasks. Future work may extend our pipeline and visual element-focused embedding and retrieval method to use with image generation models for better quality and more realistic generated images.

4 Conclusion

In this project, we present a multimodal Retrieval-Augmented Generation (RAG) pipeline with a focus on visual elements through a dual-representation framework, where text and visual features are separated. We utilize state-of-the-art pretrained foundational models (DINOv2 and Talk2DINO) as the embedder and retriever, and achieve results on par with or slightly better compared to results presented in benchmark papers. Our preliminary testing shows that MLLM-based captioning reaches 97.8% accuracy, and the DINOv2-Talk2DINO pipeline retrieves fine-grained matches, supporting our hypothesis that separate retrieval for different modalities achieves enhanced retrieval performance and improves grounding.

However, our pipeline still faces limitations in preprocessing overhead, theoretical bounds of dense retrieval, and current MLLM deficits in visual perception. Despite these limitations, our pipeline offers a solid foundation for generation grounding with retrieved visual evidence in high-stakes domains where generation accuracy is crucial.

Future work will explore more efficient preprocessing, agentic retrieval, domain-specific fine-tuning of embedders and retrievers, and extending the pipeline to text-to-vision image generation tasks.

References

- [1] S. M. T. I. Tonmoy, S. M. M. Zaman, V. Jain, A. Rani, V. Rawte, A. Chadha, and A. Das, “A comprehensive survey of hallucination mitigation techniques in large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.01313>
- [2] M. M. Abootorabi, A. Zobeiri, M. Dehghani, M. Mohammadkhani, B. Mohammadi, O. Ghahroodi, M. S. Baghshah, and E. Asgari, “Ask in any modality: A comprehensive survey on multimodal retrieval-augmented generation,” in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 16 776–16 809. [Online]. Available: <https://aclanthology.org/2025.findings-acl.861/>
- [3] S.-C. Lin, C. Lee, M. Shoeybi, J. Lin, B. Catanzaro, and W. Ping, “MM-embed: Universal multimodal retrieval with multimodal LLMs,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=i45NQb2iKO>
- [4] S. Gao, S. Zhao, X. Jiang, L. Duan, Y. X. Chng, Q.-G. Chen, W. Luo, K. Zhang, J.-W. Bian, and M. Gong, “Scaling beyond context: A survey of multimodal retrieval-augmented generation for document understanding,” 2026. [Online]. Available: <https://arxiv.org/abs/2510.15253>
- [5] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, “Learning transferable visual models from natural language supervision,” in *International conference on machine learning*. PmLR, 2021, pp. 8748–8763.
- [6] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. HAZIZA, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, “DINOv2: Learning robust visual features without supervision,” *Transactions on Machine Learning Research*, 2024, featured Certification. [Online]. Available: <https://openreview.net/forum?id=a68SUt6zFt>
- [7] Z. Bai, P. Wang, T. Xiao, T. He, Z. Han, Z. Zhang, and M. Z. Shou, “Hallucination of multimodal large language models: A survey,” 2025. [Online]. Available: <https://arxiv.org/abs/2404.18930>
- [8] S.-J. Xia, H. Zhang, Y. Jiang, X. Chen, Y. Chen, Z. Li, Z. Wan, and A. Sun, “Rethinking hallucinations: A cognitive-inspired taxonomy and comprehensive survey in large language models, large vision-language models, and multimodal large language models,” *Large Vision-Language Models, and Multimodal Large Language Models (November 09, 2025)*, 2025.
- [9] S. Leng, Y. Xing, Z. Cheng, Y. Zhou, H. Zhang, X. Li, D. Zhao, S. Lu, C. Miao, and L. Bing, “The curse of multi-modalities: Evaluating hallucinations of large multimodal models across language, visual, and audio,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025. [Online]. Available: <https://openreview.net/forum?id=G4AZhSEcrV>

- [10] L. Barsellotti, L. Bianchi, N. Messina, F. Carrara, M. Cornia, L. Baraldi, F. Falchi, and R. Cucchiara, “Talking to DINO: Bridging self-supervised vision backbones with language for open-vocabulary segmentation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2025, pp. 22 025–22 035.
- [11] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *2009 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops (CVPR Workshops)*. Los Alamitos, CA, USA: IEEE Computer Society, Jun. 2009, pp. 248–255. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/CVPR.2009.5206848>
- [12] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Proceedings of the 26th International Conference on Neural Information Processing Systems - Volume 1*, ser. NIPS’12. Red Hook, NY, USA: Curran Associates Inc., 2012, pp. 1097–1105.
- [13] B. Zhang, P. Zhang, X. Dong, Y. Zang, and J. Wang, “Long-CLIP: Unlocking the long-text capability of CLIP,” in *Computer Vision – ECCV 2024: 18th European Conference, Milan, Italy, September 29–October 4, 2024, Proceedings, Part LI*. Berlin, Heidelberg: Springer-Verlag, 2024, p. 310–325. [Online]. Available: https://doi.org/10.1007/978-3-031-72983-6_18
- [14] Y. Wu, Q. Long, J. Li, J. Yu, and W. Wang, “Visual-RAG: Benchmarking text-to-image retrieval augmented generation for visual knowledge intensive queries,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.16636>
- [15] G. Van Horn, E. Cole, S. Beery, K. Wilber, S. Belongie, and O. Mac Aodha, “Benchmarking representation learning for natural world image collections,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2021, pp. 12 884–12 893.
- [16] K. Dong, Y. Chang, S. Huang, Y. Wang, R. Tang, and Y. Liu, “Benchmarking retrieval-augmented multimodal generation for document question answering,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025. [Online]. Available: <https://openreview.net/forum?id=W4b3v9jx1p>
- [17] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, “From local to global: A graph RAG approach to query-focused summarization,” 2025. [Online]. Available: <https://arxiv.org/abs/2404.16130>
- [18] X. Ren, L. Xu, L. Xia, S. Wang, D. Yin, and C. Huang, “VideoRAG: Retrieval-augmented generation with extreme long-context videos,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.01549>
- [19] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MINILM: deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS ’20. Red Hook, NY, USA: Curran Associates Inc., 2020.
- [20] L. Gao, X. Ma, J. Lin, and J. Callan, “Precise zero-shot dense retrieval without relevance labels,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds.

- Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 1762–1777. [Online]. Available: <https://aclanthology.org/2023.acl-long.99/>
- [21] A. Bansal, “Optimizing RAG with hybrid search and contextual chunking,” *Journal of Engineering and Applied Sciences Technology*, pp. 1–5, 08 2023.
- [22] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423/>
- [23] R. Mokady, A. Hertz, and A. H. Bermano, “ClipCap: CLIP prefix for image captioning,” 2021. [Online]. Available: <https://arxiv.org/abs/2111.09734>
- [24] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, S. Wang, K. Zhang, Y. Wang, W. Gao, L. Ni, and J. Guo, “A survey on LLM-as-a-Judge,” 2025. [Online]. Available: <https://arxiv.org/abs/2411.15594>
- [25] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu, “Qwen2.5 technical report,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.15115>
- [26] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS ’20. Red Hook, NY, USA: Curran Associates Inc., 2020.
- [27] O. Weller, M. Boratko, I. Naim, and J. Lee, “On the theoretical limitations of embedding-based retrieval,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=k9CzIvzfaA>
- [28] L. Chen, W. Xie, Y. Liang, H. He, H. Zhao, Z. Yang, Z. Huang, H. Wu, H. Lu, Y. Charles, Y. Bao, Y. Fan, G. Li, H. Shen, X. Chen, W. Xu, S. Si, Z. Cai, W. Chai, Z. Huang, F. Liu, T. Liu, B. Chang, X. Hu, K. Chen, Y. Ren, Y. Liu, Y. Gong, and K. Li, “BabyVision: Visual reasoning beyond language,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.06521>
- [29] J. Y. Koh, D. Fried, and R. R. Salakhutdinov, “Generating images with multimodal language models,” in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 21 487–21 506. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/43a69d143273bd8215578bde887bb552-Paper-Conference.pdf

A Prompt Templates

For all LLM or MLLM calls, the fallback system prompt (if none specified) is as follows.

You are a helpful assistant that follows closely the following \ instructions provided by the user.

In image captioning, we provide captioning instructions and guidelines, context from the source document, and the image itself to the MLLM. The image is not part of the textual prompt.

Based on the surrounding text context and the image content, \ provide a concise, descriptive caption for this image.

Surrounding context:

{context}

Your task is to produce a caption for the image with context \ from the original source.

Describe what you see in the image, and give a comprehensive \ caption that explains the image content as well as the context of the document. However, do not give information that is too excessive in \ the caption.

Remain factual in writing the caption.

The caption should:

1. Describe the visual content clearly
2. Incorporate relevant information from the context
3. Be concise (1-2 sentences)
4. Start with "Image: "
5. NOT express that you based it off the provided context

Caption:

When the knowledge graph option is chosen, we use an LLM to extract entities and relationships from a chunk of text. The entities and relationships form nodes and edges in the knowledge graph respectively. We instruct the LLM to extract entities and relationships from the provided chunk, and output in the format of a JSON dictionary for easy handling and processing.

You are an intelligent document processor, and you will assist \

in storing document information efficiently.

The information below is to be parsed into entities and \

relationships between them.

Provided Information

{text}

Instructions

Read the information and extract entities and relationships \

between them from the text.

Entities

For entities you find in the text, provide its **name** and \

a short description of what it is.

Relationships

For relationships between two entities you find in the text, \

provide the **source entity**, **target entity**,

the **relationship** between them (relationship type), and \

their **a short description**.

Output Format

Give your output in a JSON string similar to the following.

Do **NOT** include any markdown backticks in your output.

Remember that your output will be directly parsed into \

a Python object.

Example Output

```
{{
  "entities": [
    {"name": "Entity1", "description": "Description of \
                                         entity1"}},
    {"name": "Entity2", "description": "Description of \
                                         entity2"}}
  ],
  "relationships": [
    {{
      "source": "Entity1",
      "target": "Entity2",
      "relationship": "influences",
      "description": "How entity1 influences entity2"
    }}
  ]
}}
```

Response:

Communities of nodes in the knowledge graph are detected using Louvain's algorithm. For each community, we generate a community summary using an LLM. We include instructions for summarizing and the community's information in the prompt.

You are a helpful assistant that will assist with \

data handling tasks.

Previously some documents were processed, and entities \

and their relationships were extracted to form a \

knowledge graph.

Some communities were found in the knowledge graph, \

and your help is needed in organizing these communities.

Summarize the following community of related entities \

in about {self.config.community_summary_length} words.

Main entities:

```
{", ".join(f"{node["name"]}" for node in \
            community_info["nodes"])}

```

Key relationships:

```
{"; ".join(f"{relation["source"]} {relation["relationship"]} \
            {relation["target"]}" for relation in \
            community_info["relationships"])}

```

Provide a concise summary that captures the main themes

and relationships within this community.

We use the Hypothetical Document Embeddings (HyDE) technique for text-to-text retrieval. The hypothetical document is a zero-shot generated answer to the input query. While the zero-shot answer may be highly incorrect and hallucinated, the generated text contains similar nuances and text patterns similar to retrieval targets that contain evidence to answer the question. This improves retrieval performance. We prompt an LLM with short and concise instructions, and the input query.

Based on the query, generate a detailed hypothetical \

answer document (approximately 200-300 words).

Query: {query}

Hypothetical Document:

The prompt for final answer generation contains retrieved context and the user's query, as well as instructions on the style and formation of the answer. If there are query images, or images are retrieved to be used as context, the textual prompt to provide such instructions as well.

You are a helpful assistant that answers questions \
based on retrieved context in possibly both text and visual form.

```
""" if not query_img else f"The user has provided an \  
    image in their query, and it is the first \  
    {len(query_img)} images among all images \  
    presented to you."}  
""" if not context_images else f"The last \  
    {len(context_images)} presented to you are \  
    relevant visual context to assist you in \  
    answering the question."
```

Based on the following context, please answer \
the question.

Context:

{context}

Question: {query}

Please provide a comprehensive answer.

Always trust the context over your own knowledge.

If you find the context insufficient for answering \
the question, gently reject answering the question.

You act as a chatbot and are having a conversation \

with the user.

Provide a natural response as in a conversation; \

do NOT mention anything about the context in your answer.

Answer:

In testing and benchmarking, we utilize the LLM-as-judge technique [24] to judge final answer correctness. The prompt to judge generated outputs is

You are an evaluation judge. Determine if the generated \

answer is correct given the ground truth answer(s).

Question: {question}

Generated Answer: {generated_ans}

Ground Truth Answer(s): {ground_truths_str}

The generated answer should be considered CORRECT if it:

1. Contains the same factual information as any of the \
- ground truth answers
2. Is semantically equivalent (e.g., "December 1972" \
- matches "14 December 1972 UTC")
3. Minor formatting differences are acceptable \
- (e.g., "James I" vs "James I.")

The generated answer should be considered INCORRECT if:

1. It contains wrong factual information
2. It's too vague or ambiguous

3. It contradicts the ground truth

Respond with ONLY "CORRECT" or "INCORRECT" followed \
by a brief explanation.

Format:

CORRECT/INCORRECT: <brief explanation>

B Summary of Mathematical Notations

The following is a summary of all mathematical notations used in the formal task definition.

Symbol	Type	Description
$\mathcal{D}$	Set	Set of all documents, $\mathcal{D} = \{d^{(i)}\}_{i=1}^n$
$d^{(i)}$	Document	The i -th document in the collection
$t^{(i)}$	Text	Raw text content of document i
$\mathcal{I}^{(i)}$	Set	Set of images in document i , $\mathcal{I}^{(i)} = \{m_j^{(i)}\}_{j=1}^p$
$m_j^{(i)}$	Image	The j -th image in document i
$p_j^{(i)}$	String	Caption for the j -th image in document i
$t_{m_j^{(i)}}^{(i)}$	Text	Surrounding text context of image $m_j^{(i)}$ in document i
x	Prompt string	Instruction prompt given to the MLLM for caption generation
$\mathcal{P}^{(i)}$	Set of strings	Set of generated captions for all images in document i
$\text{LLM}(\cdot)$	Function	LLM used for processing knowledge graph and communities
$\text{MLLM}(\cdot)$	Function	Multimodal LLM used for captioning and answer generation
$t_{\text{combined}}^{(i)}$	Text	Full text after inserting captions into the original text
L	Integer	Chunk size in characters
O	Integer	Overlap length in characters between consecutive chunks
$c_j^{(i)}$	Text string	The j -th text chunk of document i
m	Integer	Total number of chunks produced from a document
$\mathbf{v}_j^{(i)}$	Vector ($\mathbb{R}^d$)	Dense embedding of chunk $c_j^{(i)}$ obtained from text encoder $\mathcal{N}_{\text{text}}$
$\mathcal{N}_{\text{text}}$	Function	Neural text embedding model (MiniLM)

Symbol	Type	Description
$\mathcal{S}$	Vector database	External vector database storing all text chunk embeddings
$\mathcal{G}$	Graph	Knowledge graph (N, E) used in the graph storage option
N	Set	Nodes in the knowledge graph (extracted entities)
E	Set	Edges in the knowledge graph (relationships between entities)
C	Set of sets	Communities detected in the knowledge graph, $C = \{C_1, C_2, \dots\}$
$C_{\text{summaries}}$	Set of strings	Natural language summaries for each community, generated by an LLM
$\mathbf{g}_j^{(i)}$	Vector ($\mathbb{R}^d$)	DINOv2 embedding of image $m_j^{(i)}$
$\mathcal{S}_{\text{images}}$	Multi-vector DB	Database storing tuples $(\mathbf{g}_j^{(i)}, p_j^{(i)})$ where $p_j^{(i)}$ is the image caption
$\mathcal{Q}$	Tuple	User query, $\mathcal{Q} = (q_{\text{text}}, \mathcal{Q}_{\text{images}})$
q_{text}	String	Text part of the user query
$\mathcal{Q}_{\text{images}}$	Set of images	Images provided by the user as part of the query
k	Integer	Number of top items retrieved (e.g., top- k chunks or communities)
$\text{sim}(\cdot, \cdot)$	Function	Similarity metric (cosine similarity)
$\mathcal{C}^R$	Set of vectors	Retrieved text chunks (their embeddings) from dense retrieval
$\mathcal{I}^R$	Set of vectors	Retrieved image embeddings from dense retrieval
$\text{DINOv2}(\cdot)$	Function	Model extracting features from images
$\text{Talk2DINO}(\cdot)$	Function	Model projecting text embeddings into DINOv2 feature space
$\text{Long-CLIP}(\cdot)$	Function	Text encoder with extended context window (replaces CLIP text encoder in cross-modal retrieval)

Symbol	Type	Description
C^R	Set of communities	Retrieved communities when using graph-based retrieval
s_j	Scalar ($\mathbb{R}$)	Cosine similarity score between query embedding and community summary j
a	String	Final answer generated by the MLLM

C Retrieval and Generation Integration Tests

We obtain a second corpus of another 4 documents retrieved from Wikipedia⁷. From information on these pages, we handcraft a small dataset of 11 queries for testing end-to-end processing and generation, to verify that our pipeline works as expected.

The 11 questions are as follows:

1. What colors have been used on the livery of Fuxing trains?
2. What is the CR450 and what is special about it?
3. On which railway lines does the Fuxing train CR400 operate?
4. What colors are the Hexie trains in?
5. How many seats does the engineer's compartment on the CRH3 have?
6. What is the front of the CRH1 train like?
7. What is the colour scheme of the interior of the MTR CRRC Changchun train (also known as TML C-Train)?
8. Why is the MTR CRRC Changchun EMU (TML C-Train) compared to the SP1900, and what differences do these two trains have in terms of appearance and interior?
9. Where were the MTR CRRC Changchun EMU (TML C-Train) trainsets built? When did the trainsets fully enter service?
10. What quantitative differences do first-class carriages of the SP1900 train have as compared to other carriages?
11. What major incidents have happened to SP1900 trains?

⁷The Wikipedia pages are linked as follows:

[https://en.wikipedia.org/wiki/Fuxing_\(train\)](https://en.wikipedia.org/wiki/Fuxing_(train))

[https://en.wikipedia.org/wiki/Hexie_\(train\)](https://en.wikipedia.org/wiki/Hexie_(train))

https://en.wikipedia.org/wiki/MTR_CRRC_Changchun_EMU

https://en.wikipedia.org/wiki/MTR_SP1900_EMU

We perform zero-shot, text-only retrieval, and multimodal retrieval tests using this dataset, and experiment with different values of k for top- k retrieval. The results in answer accuracies are as follows, with tuples representing (dense retrieval, graph retrieval) scores.

Value of k	Text-only	Multimodal
0 (Zero-shot)	27.3	–
1	36.4, 9.1	45.5 , 18.2
3	36.4, 18.2	45.5 , 45.5
5	45.5 , 18.2	27.3, 27.3
7	36.4, 9.1	54.6 , 9.1
10	45.5, 18.2	54.6 , 36.4
15	45.5 , 9.1	27.3, 18.2
20	45.5 , 9.1	36.4, 9.1

D Project Planning

D.1 GANTT Chart

Task	Jul	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr
Do the literature survey										
Implement simple text-based RAG										
Collect data from online sources										
Test CLIP-based embedding models										
Design and implement embedding method										
Test out graph-based storage methods										
Design and implement storage methods										
Define retrieval criteria										
Implement retrieval based on query										
Test embedding effectiveness										
Test retrieval effectiveness										
Test with actual LLM / MLLM										
Write the proposal										
Write the monthly reports										
Write the progress report										
Write the final report										
Prepare for the oral defense										
Prepare the project video										

E Required Hardware and Software

E.1 Hardware

Development PC: a Unix-based system.

E.2 Software

The following Python packages are required.

Python Dependencies

- pandas, numpy: data handling and numerical operations
- PyMuPDF, python-docx: PDF and DOCX file processing
- torch, torchvision, Pillow: deep learning and image processing
- sentence-transformers, transformers: text embeddings and transformer models
- langchain-text-splitters: text chunking for retrieval
- chromadb: vector database
- networkx, python-louvain: knowledge graphs and community detection
- openai, google-generativeai, dashscope: access to GenAI models
- DINOv2 dependencies: advanced image feature extraction
 - torch, torchvision, torchmetrics
 - omegaconf
 - fvcore
 - iopath
 - xformers
 - cuml-cu11
 - submitit
- Talk2DINO dependencies: vision-language cross-modal alignment and interaction
 - matplotlib
 - opencv-python
 - scikit-image

- timm
 - pyyaml
 - requests
 - tqdm
 - einops
 - webdataset
 - numpy
- CLIP: multimodal embeddings